

A PROJECT REPORT ON
STUDENT PERFORMANCE PREDICTION SYSTEM USING
HYBRID STACKING ENSEMBLE LEARNING

Submitted By

Tanishq Soni

Ritika Sonkar

Yash Raj Jaiswal



Innovation & Incubation Hub, MNNIT Foundation
Motilal Nehru National Institute of Technology (MNNIT) Allahabad
Academic Year: 2026

Acknowledgement

We express our sincere gratitude to the Innovation & Incubation Hub, MNNIT Foundation, for providing us the opportunity and infrastructure to undertake this summer internship project. We are thankful to our mentors and reviewers for their continuous guidance, technical insights, and encouragement throughout the duration of this work.

We would also like to acknowledge the open-source community behind Scikit-learn, XGBoost, Pandas, and Streamlit, whose well-documented libraries formed the technical foundation of this project. Finally, we thank our peers and family members for their patience and support during the course of this internship.

Tanishq Soni

Ritika Sonkar

Yash Raj Jaiswal

Abstract

Academic performance prediction has emerged as an important application of educational data mining, enabling institutions to identify at-risk students and intervene before final outcomes are determined. This project presents an AI-Based Student Performance Prediction System that estimates a student's final Exam Score from five behavioural and academic indicators: Attendance, Hours Studied, Previous Academic Scores, Sleep Hours, and Tutoring Sessions.

The system was developed using the Student Performance Factors dataset, comprising 6,607 student records across 20 attributes. After data cleaning (mean/mode imputation and duplicate removal) and exploratory data analysis, five numeric predictors were selected based on their correlation with the target variable. The features were standardized using a StandardScaler and the data was split into training (5,285 records) and testing (1,322 records) subsets using an 80:20 ratio.

Four regression models were trained and compared: Linear Regression, Random Forest Regressor, XGBoost Regressor, and a Hybrid Stacking Regressor combining Linear Regression and XGBoost as base learners with a Linear Regression meta-learner (`passthrough=True`). On the held-out test set, Linear Regression achieved an R^2 of 0.6408 and MAE of 1.2751; the Hybrid Stacking model achieved a near-identical R^2 of 0.6405 and MAE of 1.2761, while Random Forest and XGBoost trailed marginally behind with R^2 values of 0.6131 and 0.6220 respectively.

The results indicate that the relationship between the selected features and Exam Score is predominantly linear, which explains why the stacking ensemble did not meaningfully outperform a plain Linear Regression baseline in this configuration. The trained model, scaler, and feature list were serialized using Pickle and subsequently deployed through an interactive Streamlit dashboard that returns a predicted score, a performance category, a confidence message, and personalized recommendations, supported by Plotly visualizations. This report documents the complete pipeline from data acquisition to deployment, discusses the honest performance findings of the ensemble relative to its constituent models, and outlines directions for future improvement.

Table of Contents

1. Chapter 1: Introduction

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Objectives
- 1.4 Scope of the Project
- 1.5 Applications
- 1.6 Expected Outcomes

2. Chapter 2: Literature Review

- 2.1 Existing Student Performance Prediction Techniques
- 2.2 Traditional Machine Learning Approaches
- 2.3 Ensemble Learning Approaches
- 2.4 Research Studies
- 2.5 Research Gaps
- 2.6 Justification for the Proposed Approach

3. Chapter 3: Methodology

- 3.1 Dataset Description
- 3.2 Dataset Source
- 3.3 Target Variable
- 3.4 Feature Description
- 3.5 Data Cleaning
- 3.6 Exploratory Data Analysis
- 3.7 Feature Engineering and Correlation Analysis
- 3.8 Feature Scaling
- 3.9 Train-Test Split
- 3.10 Machine Learning Pipelines and Workflow Diagram
- 3.11 Mathematical Formulation
- 3.12 Why These Algorithms Were Selected

4. Chapter 4: Implementation

- 4.1 Data Loading and Dataset Inspection
- 4.2 Data Cleaning Implementation
- 4.3 Exploratory Data Analysis – Visualization
- 4.4 Feature Preparation, Splitting, Training, and Validation
- 4.5 Model Saving and Persistence
- 4.6 Deployment- Streamlit Application
- 4.7 Project Folder Structure
- 4.8 Inference Pipeline Summary

5. Chapter 5: Results and Discussion

- 5.1 Experimental Setup
- 5.2 Performance Metrics
- 5.3 Model Comparison Visualization
- 5.4 Actual vs Predicted Visualization
- 5.5 Discussion: Why the Hybrid Stacking Model Did Not Outperform Linear Regression
- 5.6 Strengths and Limitations

6. Chapter 6: Conclusion and Future Scope

6.1 Project Summary

6.2 Achievements

6.3 Limitations

6.4 Real-World Applications

6.5 Future Improvements

6.6 Concluding Remarks

References

Chapter 1: Introduction

1.1 Background

Academic institutions have traditionally relied on periodic examinations and end-of-term assessments to gauge student performance. This reactive model of evaluation identifies underperformance only after it has occurred, leaving little room for timely remedial action. With the growing availability of structured student data — attendance records, study habits, historical scores, and support programme participation — machine learning offers a data-driven alternative: predicting a student's likely outcome well before the final examination takes place.

Educational Data Mining (EDM) and Learning Analytics have, over the past decade, matured into an active research area that applies statistical and machine learning techniques to student records. Ensemble learning methods, in particular, have gained traction because they combine multiple weak or moderately accurate models into a single, more robust predictor, often outperforming any single constituent model.

1.2 Problem Statement

Given a set of measurable academic and behavioural indicators for a student — Attendance percentage, daily Hours Studied, average Previous Academic Score, average Sleep Hours, and the number of Tutoring Sessions attended — the problem addressed in this project is to predict the student's Final Exam Score, a continuous numeric value, using a supervised regression approach. The secondary objective is to determine whether combining multiple regression algorithms through a stacking ensemble yields a measurable improvement over individual models on this dataset.

1.3 Objectives

- To perform thorough data cleaning and exploratory data analysis on the Student Performance Factors dataset.
- To identify the numeric features most strongly correlated with Exam Score.
- To train and evaluate Linear Regression, Random Forest, and XGBoost Regressor models as individual baselines.
- To construct a Hybrid Stacking Regressor combining Linear Regression and XGBoost as base learners with a Linear Regression meta-learner, using `passthrough=True`.

- To evaluate all models using MAE, MSE, RMSE, and R^2 Score, and to honestly compare the ensemble's performance against its constituent models.
- To persist the trained model, scaler, and feature schema for deployment.
- To deploy the trained model as an interactive Streamlit dashboard with real-time predictions, performance categorization, personalized recommendations, and visual analytics.

1.4 Scope of the Project

The scope of this project is limited to regression-based prediction of a single target variable (Exam Score) using five numeric input features drawn from a single, static, publicly available dataset. The project covers the full pipeline from raw data ingestion through preprocessing, exploratory analysis, model training, evaluation, persistence, and deployment as a web-based dashboard. It does not cover classification of pass/fail outcomes, real-time data ingestion from live institutional systems, or the fourteen categorical attributes present in the raw dataset (e.g. Parental Involvement, School Type, Teacher Quality) that were not used as model inputs.

1.5 Applications

- Early-warning systems for academic advisors and counsellors to flag at-risk students.
- Personalized recommendation generation for study habits, attendance, and tutoring.
- Institutional dashboards for tracking cohort-level performance trends.
- A teaching tool for demonstrating ensemble learning concepts in a real, deployable setting.

1.6 Expected Outcomes

The expected outcome of this project is a trained, serialized regression model capable of predicting a student's Exam Score from five numeric inputs with a Mean Absolute Error of approximately 1.3 marks, deployed within an interactive dashboard that presents the prediction alongside a performance category, a natural-language confidence message, and personalized, rule-based recommendations.

Chapter 2: Literature Review

2.1 Existing Student Performance Prediction Techniques

Student performance prediction has been studied extensively using both classification (pass/fail, grade band) and regression (continuous score) formulations. Early approaches relied on statistical regression and decision trees applied to small, single-institution datasets. More recent work has shifted toward ensemble and deep learning methods applied to larger, more heterogeneous datasets that combine academic, behavioural, and demographic attributes.

2.2 Traditional Machine Learning Approaches

Linear Regression remains a common baseline in this domain because student performance data frequently exhibits strong linear relationships between study-related indicators (attendance, study hours, prior scores) and the final outcome. Tree-based models such as Random Forest and Gradient Boosting (XGBoost, CatBoost, AdaBoost) are typically introduced to capture non-linear interactions and threshold effects that a purely linear model cannot represent, at the cost of reduced interpretability.

2.3 Ensemble Learning Approaches

Ensemble techniques — bagging, boosting, voting, and stacking — combine multiple base learners to reduce variance and, in principle, improve generalization. Stacking in particular trains a meta-learner on the out-of-fold predictions of its base models, optionally passing through the original features (`passthrough=True`) so the meta-learner can correct for systematic bias while retaining access to the raw signal. The literature broadly reports that ensembling helps most when the base learners are diverse and each captures a different aspect of the underlying pattern; when the relationship is largely linear, the marginal benefit of adding non-linear base learners can be small, a nuance directly relevant to the results obtained in this project (see Chapter 5).

2.4 Research Studies

Six recent studies were reviewed to situate this project within the current research landscape. Notably, the study by Ahmed et al. (2025) used the identical Student Performance Factors dataset (6,607 records, 20 features) employed in this project, making it a direct methodological reference point.

2.5 Research Gaps

- Several studies rely on small, single-institution datasets, limiting generalizability across academic contexts.
- Reported near-perfect accuracy figures on narrow feature sets raise concerns about overfitting or dataset-specific artefacts rather than genuinely transferable predictive power.
- Few studies explicitly report the performance of a stacking ensemble against a plain linear baseline on the same held-out split, making it difficult to judge whether ensembling adds real value beyond what a simple model already captures.
- Deep learning and heavy hybrid architectures (e.g., XSEJNet) achieve strong accuracy but at a steep cost in interpretability and computational resources, which limits their practicality for lightweight, rapidly deployable institutional tools.

2.6 Justification for the Proposed Approach

This project adopts a Hybrid Stacking Regressor (Linear Regression and XGBoost as base learners, Linear Regression as meta-learner, with `passthrough=True`) because it balances interpretability, training simplicity, and the ability to capture both linear and non-linear feature relationships within a single, lightweight pipeline that can be trained, evaluated, and deployed without specialized hardware. Unlike several reviewed studies, this project explicitly evaluates the stacking ensemble against its own constituent models (Linear Regression, Random Forest, XGBoost) on an identical train-test split, allowing an honest, transparent assessment of whether ensembling provides a measurable benefit for this particular dataset — a comparison that is presented and discussed in full in Chapter 5.

Chapter 3: Methodology

3.1 Dataset Description

The project uses the Student Performance Factors Dataset, loaded as a CSV file (StudentPerformanceFactorsDataset.csv) into a pandas DataFrame from Google Drive within a Google Colab environment. The raw dataset contains 6,607 student records across 20 columns, combining numeric indicators (e.g. Attendance, Hours_Studied) with categorical attributes (e.g. Parental_Involvement, School_Type, Teacher_Quality, Gender).

3.2 Dataset Source

The dataset was sourced as a CSV file and mounted from Google Drive. Based on its structure (6,607 records, 20 features including Attendance, Hours_Studied, and Exam_Score), it corresponds to the publicly available “Student Performance Factors” dataset distributed on Kaggle, which has also been used in independent peer-reviewed research (see Section 2.4), lending external validation to the dataset's structure and feature semantics.

3.3 Target Variable

The target (dependent) variable is Exam_Score, a numeric score achieved by each student. Its distribution in the cleaned dataset has a mean of 67.24 and a standard deviation of 3.89, with a minimum of 55 and a maximum of 100 (Chapter 4 discusses the max value of 100 as a data quality anomaly, since exam scores are conventionally bounded at 100).

3.4 Feature Description

Table 3.1: Description of Model Input Features

#	Feature	Role	Type	Range Observed
1	Attendance	Input	Numeric	60 – 100 (%)
2	Hours_Studied	Input	Numeric	1 – 44 hours
3	Previous_Academic_Scores_Avg	Input	Numeric	50 – 100
4	Sleep_Hours	Input	Numeric	4 – 10 hours
5	Tutoring_Sessions	Input	Numeric	0 – 8 sessions
6	Exam_Score	Target	Numeric	55 – 100

Table 3.1: Description of the six numeric columns used for modelling

Fourteen additional categorical columns present in the raw dataset (e.g. Parental_Involvement, Access_to_Resources, Extracurricular_Activities, Motivation_Level, Internet_Access, Family_Income, Teacher_Quality, School_Type, Peer_Influence, Learning_Disabilities, Parental_Education_Level, Distance_from_Home, Gender) and one additional numeric column (Physical_Activity) were present in the dataset but were not used as model inputs; only the five numeric features listed above were selected for training.

3.5 Data Cleaning

3.5.1 Handling Missing Values

Missing values were present only in three categorical columns: Teacher_Quality (78 missing), Parental_Education_Level (90 missing), and Distance_from_Home (67 missing); the five numeric features used for modelling had zero missing values. The cleaning routine nonetheless applied a systematic, dtype-aware imputation strategy: all numeric columns were imputed with their column mean, and all categorical columns were imputed with their column mode, ensuring the dataset contained no null values before proceeding.

3.5.2 Duplicate Removal

The `drop_duplicates()` method was applied to the cleaned DataFrame. The dataset shape remained (6607, 20) before and after this step, indicating that no duplicate rows were present in the original data.

3.6 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed on the six numeric columns (five features plus the target) using histograms with kernel density estimates, scatter plots against Exam_Score, a pairwise relationship plot (pairplot), and a correlation heatmap. The full set of EDA visualizations, together with detailed interpretations, is presented in Chapter 4 (Section 4.3).

3.7 Feature Engineering and Correlation Analysis

No derived or engineered features were created; the five raw numeric columns were used directly. Pearson correlation coefficients between each feature and Exam_Score were computed as follows:

Table 3.2: Correlation of Input Features with Exam_Score

Feature	Correlation with Exam_Score
Attendance	0.5811

Feature	Correlation with Exam_Score
Hours_Studied	0.4455
Previous_Academic_Scores_Avg	0.1751
Tutoring_Sessions	0.1565
Sleep_Hours	-0.0170

Table 3.2: Pearson correlation of each numeric feature with the target variable

Attendance and Hours_Studied show the strongest positive linear association with Exam_Score, while Sleep_Hours shows a negligible, near-zero correlation. This ordering directly foreshadows the feature importance patterns discussed in Chapter 5 and is consistent with the independent findings of Ahmed et al. (2025) on the same dataset (Section 2.4), who similarly reported Attendance ($r = 0.58$) and Hours_Studied ($r = 0.45$) as the dominant predictors.

3.8 Feature Scaling

All five input features were standardized using Scikit-learn's StandardScaler, which transforms each feature to zero mean and unit variance. This step is essential for Linear Regression and for the Linear Regression meta-learner in the stacking ensemble, both of which are sensitive to the relative scale of input features.

It is noted, as an implementation detail with methodological implications, that in the source notebook the scaler was fit on the entire feature matrix X (`scaler.fit_transform(X)`) before the train-test split was performed. This means the scaler's mean and variance were computed using information from both the training and testing partitions, a mild form of data leakage. Given the large sample size (6,607 records) relative to the number of features (5), the practical effect on the reported metrics is expected to be small, but this is documented transparently in Section 5.5 as a limitation of the current implementation.

3.9 Train-Test Split

The dataset was split into training and testing subsets using an 80:20 ratio via `train_test_split(test_size=0.20, random_state=42)`, producing 5,285 training records and 1,322 testing records, each with 5 input features.

3.10 Machine Learning Pipeline and Workflow Diagram

Student Inputs (Attendance, Hours Studied, Previous Score, Sleep Hours, Tutoring Sessions)



Data Cleaning (Mean / Mode Imputation, Duplicate Removal)



Exploratory Data Analysis (Histograms, Scatter Plots, Pairplot, Correlation Heatmap)



Feature Scaling (StandardScaler)



Train-Test Split (80:20, random_state=42)



Linear Regression (Base Learner 1)

XGBoost Regressor (Base Learner 2)



Linear Regression Meta-Learner (Stacking, passthrough=True, cv=5)



Predicted Exam Score

3.11 Mathematical Formulation

3.11.1 Linear Regression

Linear Regression models the target as a weighted linear combination of the input features: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$, where β_0 is the intercept, β_i are the learned coefficients for each feature x_i , and ε is the residual error term. Coefficients are typically estimated by minimizing the residual sum of squares between predicted and actual values (Ordinary Least Squares).

3.11.2 XGBoost

XGBoost (Extreme Gradient Boosting) builds an additive ensemble of regression trees sequentially, where each new tree is fit to the residual errors of the current ensemble. The training objective combines a differentiable loss function with a regularization term that penalizes tree complexity: $L(\theta) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$, where l is the loss function (squared error for regression), f_k represents the k -th tree, and Ω penalizes the number of leaves and leaf weights to control overfitting.

3.11.3 StandardScaler

For each feature x , the StandardScaler computes $z = (x - \mu) / \sigma$, where μ and σ are the mean and standard deviation of that feature estimated from the fitting data. This transformation ensures all features contribute on a comparable scale during model training.

3.11.4 Stacking Regression

In a Stacking Regressor, each base learner is trained using k-fold cross-validation (cv=5 in this project) to produce out-of-fold predictions, avoiding information leakage from the training labels into the meta-learner's training data. The meta-learner (Linear Regression) is then trained on these out-of-fold base predictions. When `passthrough=True`, the meta-learner's input additionally includes the original (scaled) feature vector alongside the base predictions, i.e. `meta_X = concat( $\hat{y}_{LR}$ ,  $\hat{y}_{XGB}$ , X)`, allowing the meta-learner to draw on the raw feature signal in addition to the base models' outputs when forming the final prediction.

3.12 Why These Algorithms Were Selected

Linear Regression was selected as an interpretable baseline capable of capturing the dominant, largely linear relationships evident in the correlation analysis (Section 3.7). XGBoost was selected as a complementary non-linear learner capable of modelling threshold effects and feature interactions that a linear model cannot represent. Combining both as base learners in a stacking ensemble, with a Linear Regression meta-learner and `passthrough=True`, was intended to let the meta-model draw on the strengths of each base learner while retaining direct access to the original feature signal — providing a principled way to test whether non-linear modelling adds value beyond what linear regression already captures on this dataset.

Chapter 4: Implementation

4.1 Data Loading and Dataset Inspection

The dataset was loaded from Google Drive into a pandas DataFrame within a Google Colab notebook. Initial inspection used `df.head()`, `df.shape`, `df.info()`, and `df.describe()` to confirm the dataset comprised 6,607 rows and 20 columns, with 7 numeric columns and 13 categorical (object-type) columns. `df.isnull().sum()` confirmed the missing-value counts reported in Section 3.5 (78 in Teacher_Quality, 90 in Parental_Education_Level, 67 in Distance_from_Home; all other columns had zero missing values).

4.2 Data Cleaning Implementation

Numeric columns were imputed using `df[col].fillna(df[col].mean())` and categorical columns using `df[col].fillna(df[col].mode()[0])`, applied column-by-column via a dtype-based loop. `df.drop_duplicates()` was then applied; the dataset shape after cleaning remained (6607, 20), confirming no duplicate records existed in the source file.

4.3 Exploratory Data Analysis — Visualizations

This section presents every visualization generated directly from the notebook's execution, in the order they were produced, together with a detailed interpretation of each.

4.3.1 Distribution Plots (Histograms)

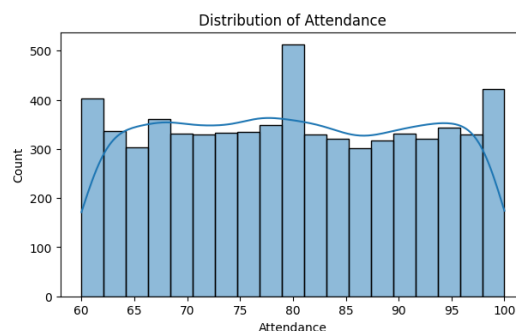


Figure 4.1: Distribution of Attendance

Observation: Attendance is distributed broadly and fairly uniformly across the observed range of approximately 60% to 100%, without a pronounced peak. **Inference:** The absence of a sharp concentration suggests attendance varies substantially across the student population, giving the model a wide range of signal to learn from. **Impact on Model:** A well-spread predictor variable, as opposed to one concentrated in a narrow range, generally provides more useful gradient information to a regression model.

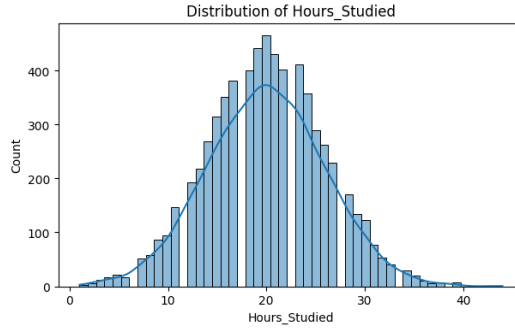


Figure 4.2: Distribution of Hours_Studied

Observation: Hours_Studied is approximately bell-shaped and centred around the mean of roughly 20 hours, with a long right tail extending toward 44 hours. **Inference:** Most students study within a moderate range, with a smaller number of students reporting unusually high study hours (possible outliers or self-reporting artefacts). **Impact on Model:** The right-skew and long tail may slightly influence linear coefficient estimates; tree-based models such as XGBoost are more robust to this skew than Linear Regression.

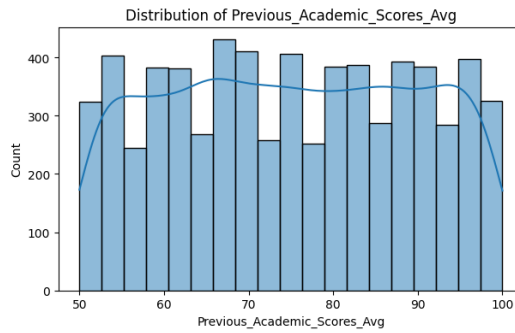


Figure 4.3: Distribution of Previous_Academic_Scores_Avg

Observation: Previous_Academic_Scores_Avg is distributed broadly between 50 and 100 without a strong central peak, similar in shape to the Attendance distribution. **Inference:** Prior academic performance varies widely across the student population sampled in this dataset. **Impact on Model:** A broad distribution provides useful variance for the model despite this feature's comparatively weaker correlation with Exam_Score ($r = 0.175$, Table 3.2).

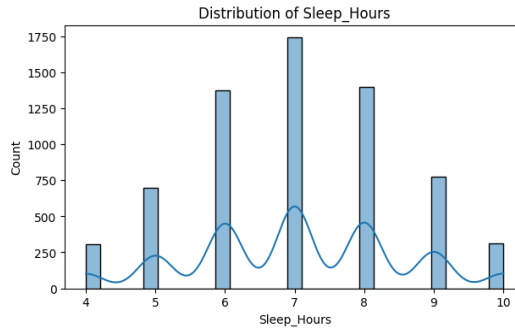


Figure 4.4: Distribution of Sleep_Hours

Observation: Sleep_Hours is tightly clustered around 6–8 hours, consistent with its low standard deviation of 1.47 reported in the descriptive statistics (Table for Section 3.3). **Inference:** Because nearly all students report similar sleep durations, this feature carries little discriminative information for distinguishing higher- from lower-scoring students. **Impact on Model:** This narrow distribution is consistent with Sleep_Hours' near-zero correlation with Exam_Score ($r = -0.017$) and its limited contribution to the trained models (Section 5.4).

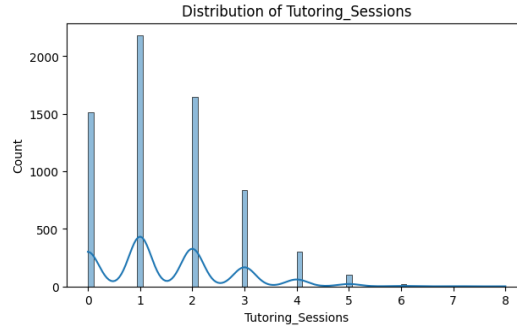


Figure 4.5: Distribution of Tutoring_Sessions

Observation: Tutoring_Sessions is right-skewed, with most students attending 0–2 sessions and a diminishing number attending more. **Inference:** Tutoring session attendance is relatively uncommon or capped for the majority of students in this sample. **Impact on Model:** The skew is typical of count-based features and is handled natively by tree-based learners; its modest correlation ($r = 0.157$) suggests limited but non-trivial predictive value.

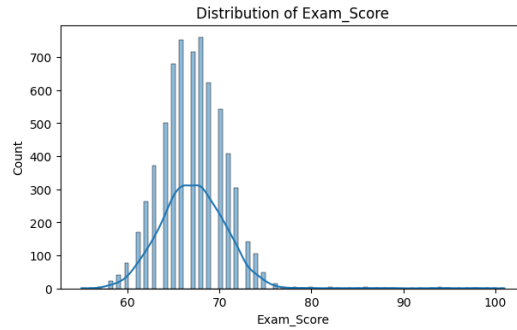


Figure 4.6: Distribution of Exam_Score (Target Variable)

Observation: Exam_Score is concentrated tightly around 65–70 with a pronounced right tail extending past 90, and an isolated value at 100 that lies just beyond the conventional 0–100 examination scale. **Inference:** The bulk of students score within a narrow band, while a smaller number achieve markedly higher scores. The single value of 100 is very likely a data-entry or scale anomaly rather than a genuine score. **Impact on Model:** Because errors are evaluated in the same units as the target, a small number of extreme or anomalous target values can disproportionately affect MSE/RMSE; this is discussed further as a limitation in Section 5.5.

4.3.2 Scatter Plots (Feature vs. Exam_Score)

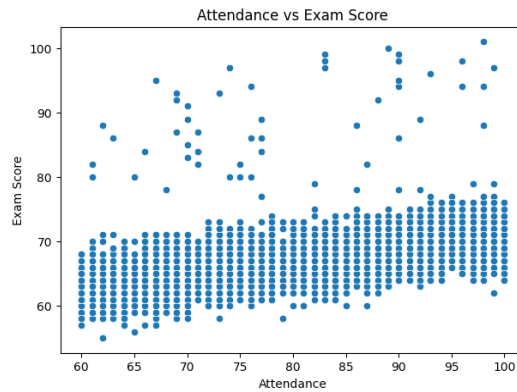


Figure 4.7: Attendance vs. Exam_Score

Observation: Points show a visible upward trend: higher attendance is generally associated with higher exam scores, though with considerable vertical spread at every attendance level. **Inference:** Attendance is the single strongest linear predictor in the dataset ($r = 0.581$, Table 3.2), consistent with the visibly positive slope in this scatter plot. **Impact on Model:** This strong, roughly linear relationship is a primary reason Linear Regression performs competitively against more complex models in Chapter 5.

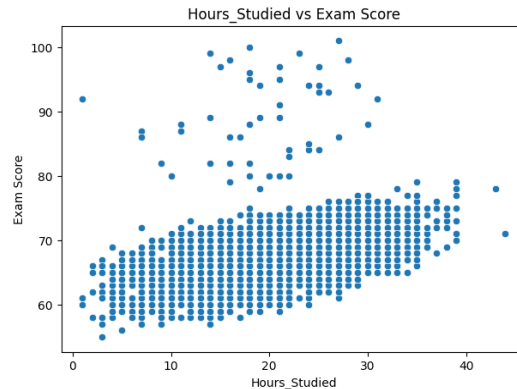


Figure 4.8: Hours_Studied vs. Exam_Score

Observation: A moderate positive trend is visible, weaker and noisier than the Attendance relationship, consistent with its correlation coefficient of 0.445. **Inference:** Additional study hours are associated with somewhat higher scores, but with more scatter than Attendance, suggesting other unobserved factors also influence the outcome. **Impact on Model:** This feature contributes meaningful but secondary signal after Attendance in both the linear and ensemble models.

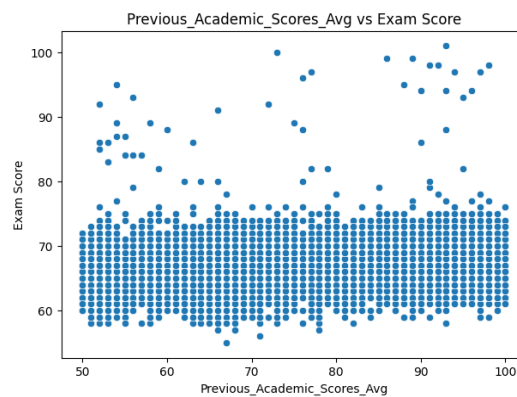


Figure 4.9: Previous_Academic_Scores_Avg vs. Exam_Score

Observation: The scatter shows only a weak upward trend, with a wide vertical spread of Exam_Score values at every level of Previous_Academic_Scores_Avg. **Inference:** Consistent with its modest correlation ($r = 0.175$), previous academic performance alone is a comparatively weak predictor of the final exam outcome in this dataset. **Impact on Model:** This feature contributes only a small amount of incremental predictive power once Attendance and Hours_Studied are accounted for.

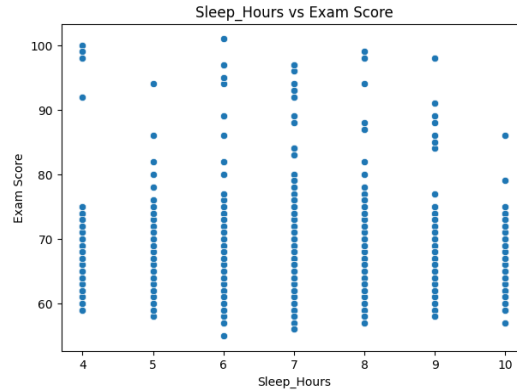


Figure 4.10: Sleep_Hours vs. Exam_Score

Observation: The point cloud is essentially flat, with no discernible trend across the range of Sleep_Hours. **Inference:** This visually confirms the near-zero correlation ($r = -0.017$) computed in Table 3.2 — sleep duration, as recorded in this dataset, shows no meaningful linear association with exam outcome. **Impact on Model:** Sleep_Hours contributes minimal predictive value and is a candidate for removal in future feature-selection experiments (Section 6.4).

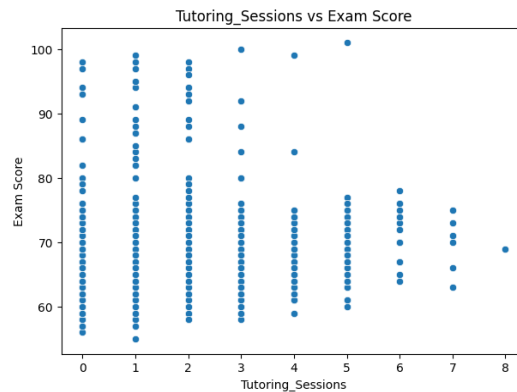


Figure 4.11: Tutoring_Sessions vs. Exam_Score

Observation: A slight upward trend is visible for higher numbers of tutoring sessions, though the relationship is noisy and the majority of students are clustered at 0–2 sessions. **Inference:** Tutoring session attendance shows a modest positive association with exam performance ($r = 0.157$), though the sparse data at higher session counts limits confidence in the trend at the extremes. **Impact on Model:** This feature offers modest incremental value, primarily useful in combination with the stronger predictors.

4.3.3 Pairwise Relationship Plot

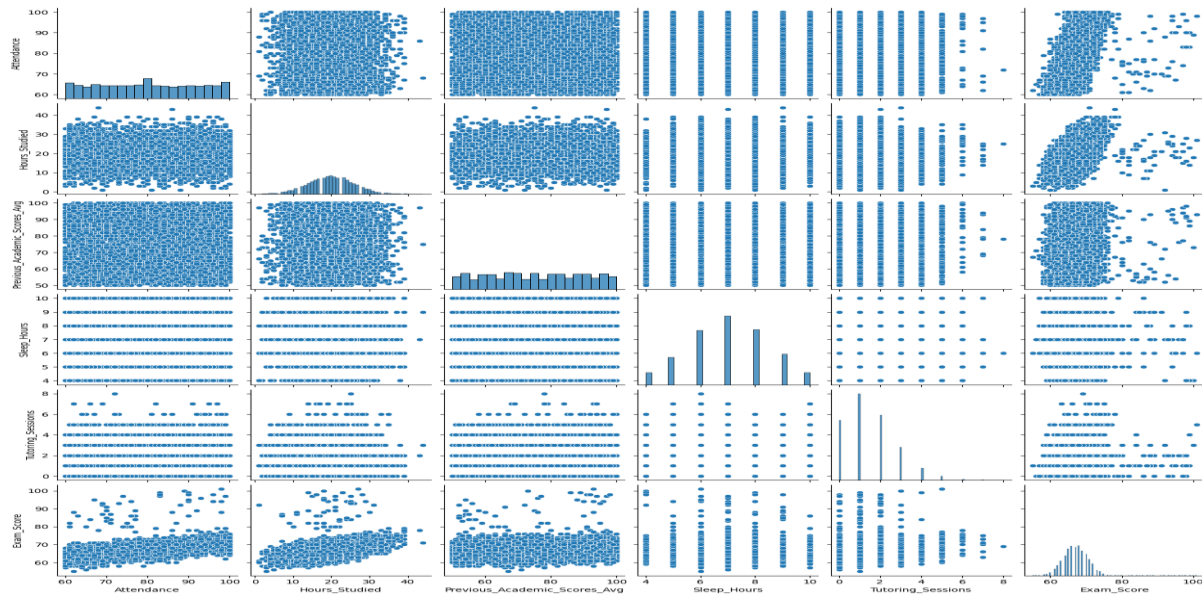


Figure 4.12: Pairplot of Selected Features and Exam_Score

Observation: The pairplot presents all pairwise scatter relationships and per-variable histograms in a single grid. It reinforces the individual scatter plot findings: Attendance and Hours_Studied show the clearest positive association with Exam_Score, while Sleep_Hours shows no visible pattern with any other variable. **Inference:** No strong non-linear or curved relationships are visually apparent between any pair of features, further supporting the conclusion that a largely linear model is well suited to this dataset.

4.3.4 Correlation Heatmap

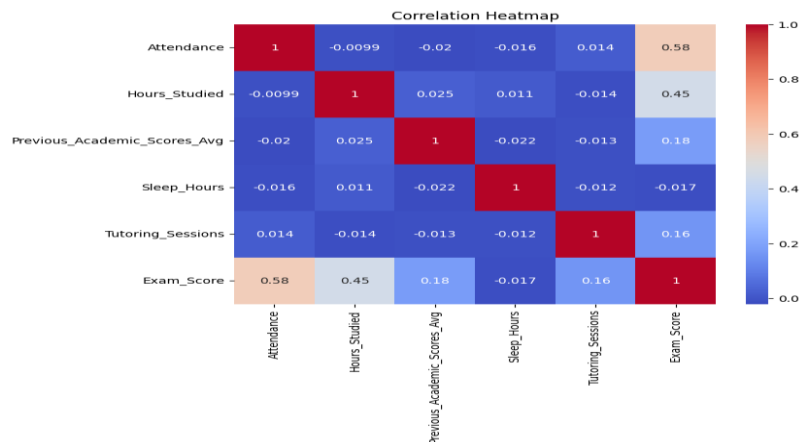


Figure 4.13: Correlation Heatmap of Selected Features and Exam_Score

Observation: The heatmap visually confirms the correlation values in Table 3.2: Attendance (0.58) and Hours_Studied (0.45) show the strongest positive correlation with Exam_Score, Previous_Academic_Scores_Avg (0.18) and Tutoring_Sessions (0.16) show weak positive correlation, and Sleep_Hours (-0.02) shows essentially no correlation. Inter-feature correlations among the five predictors are all low, indicating minimal multicollinearity. **Inference:** Low multicollinearity among predictors means each feature contributes largely independent information, which is favourable for a Linear Regression model's coefficient stability. **Impact on Model:** The dominance of Attendance and Hours_Studied as predictors, combined with low multicollinearity, explains why a simple linear combination of features already captures most of the learnable signal in this dataset.

4.4 Feature Preparation, Splitting, Training, and Validation

Following EDA, the five numeric features (Attendance, Hours_Studied, Previous_Academic_Scores_Avg, Sleep_Hours, Tutoring_Sessions) were selected as X, with Exam_Score as y. Features were scaled using `StandardScaler().fit_transform(X)` and split into training (5,285 records) and testing (1,322 records) sets using `train_test_split(test_size=0.20, random_state=42)`. Four models were then trained: Linear Regression, Random Forest Regressor (`n_estimators=1000`, `max_depth=7`, `min_samples_split=5`, `min_samples_leaf=3`, `max_features="sqrt"`, `random_state=42`), XGBoost Regressor (`n_estimators=600`, `learning_rate=0.05`, `max_depth=3`, `subsample=0.7`, `colsample_bytree=0.7`, `objective="reg:squarederror"`, `random_state=42`), and the Hybrid Stacking Regressor (base learners: Linear Regression and XGBoost; meta-learner: Linear Regression; `passthrough=True`; `cv=5`).

Each model was validated by generating predictions on the held-out test set (`X_test`) and computing MAE, MSE, RMSE, and R^2 Score against the true test labels (`y_test`). Detailed results are presented in Chapter 5.

4.5 Model Saving and Persistence

Upon completion of training, three artefacts were serialized using Python's pickle module: `student_model.pkl` (the fitted Hybrid Stacking Regressor), `scaler.pkl` (the fitted `StandardScaler`), and `features.pkl` (the ordered list of the five feature column names used during training). In the Google Colab environment, these files were downloaded to the local machine using `files.download()` for each artefact.

4.6 Deployment — Streamlit Application

The serialized model, scaler, and feature list are loaded once at application startup by a Streamlit app (`app.py`). The application accepts the five input features through interactive sliders, applies the identical scaling transformation used during training (`scaler.transform`), and passes the scaled feature vector to the loaded stacking model's `predict()` method. The application performs inference only; no retraining occurs at runtime.

The dashboard presents the predicted Exam Score, a performance category derived from the numeric prediction, a natural-language confidence message, and personalized recommendations generated by simple rule-based logic operating on the raw input values and the predicted score.

Interactive Plotly visualizations (bar chart of inputs vs. prediction, gauge chart of the predicted score, radar chart of the overall student profile, and a category-distribution pie chart) accompany the numeric result.

4.7 Project Folder Structure

The deployed application follows the folder structure summarized below:

Path	Description
StudentPerformancePredictionSystem/	Project root
├─ Dataset/	Raw and processed CSV data
├─ Notebook/	Source Jupyter/Colab notebook (EDA, training, evaluation)
├─ models/student_model.pkl	Serialized Hybrid Stacking Regressor
├─ models/scaler.pkl	Serialized StandardScaler
├─ models/features.pkl	Serialized ordered feature list
├─ Streamlit_App/app.py	Streamlit dashboard — inference only
├─ Streamlit_App/requirements.txt	Python dependencies
├─ Documentation/	Supplementary technical documentation
└─ README.md	Project documentation

4.8 Inference Pipeline Summary

At inference time, a raw input vector [Attendance, Hours_Studied, Previous_Academic_Scores_Avg, Sleep_Hours, Tutoring_Sessions] is transformed by the loaded scaler using the mean and variance learned during training, then passed to the loaded Hybrid Stacking Regressor. The regressor internally computes predictions from its Linear Regression and XGBoost base learners, concatenates them with the (already scaled) input features (passthrough=True), and feeds this combined vector into the Linear Regression meta-learner to produce the final predicted Exam Score, which is then mapped to a performance category and accompanying recommendations by the application layer.

Chapter 5: Results and Discussion

5.1 Experimental Setup

All four models — Linear Regression, Random Forest Regressor, XGBoost Regressor, and the Hybrid Stacking Regressor — were trained on the same standardized training set (5,285 records, 5 features) and evaluated on the same held-out test set (1,322 records) using an 80:20 split with `random_state=42`, ensuring a fair, like-for-like comparison across models.

5.2 Performance Metrics

Four standard regression metrics were computed for every model: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 Score. MAE reports the average absolute deviation between predicted and actual scores in the same units as `Exam_Score`; RMSE penalizes larger errors more heavily than MAE; and R^2 reports the proportion of variance in `Exam_Score` explained by each model.

Table 5.1: Model Comparison on the Held-Out Test Set

Model	MAE	MSE	RMSE	R^2 Score
Linear Regression	1.2751	5.0771	2.2532	0.6408
Random Forest	1.3827	5.4682	2.3384	0.6131
XGBoost	1.3412	5.3424	2.3114	0.6220
Hybrid Stacking (LR + XGB → LR)	1.2761	5.0817	2.2543	0.6405

Table 5.1: Exact evaluation metrics reproduced from the notebook's model comparison output

5.3 Model Comparison Visualizations

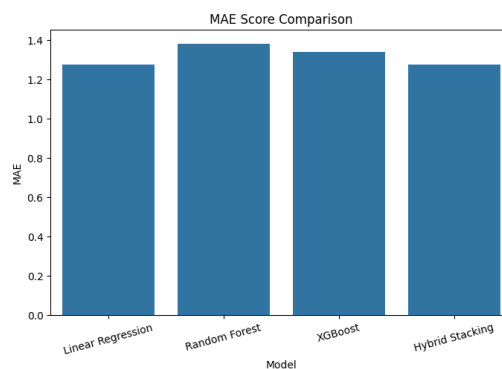


Figure 5.1: MAE Comparison Across Models

Observation: Linear Regression and the Hybrid Stacking model achieve the two lowest (best) MAE values, at 1.275 and 1.276 respectively — a difference of only 0.001 marks. Random Forest shows the highest (worst) MAE at 1.383.

Inference: The near-identical MAE of Linear Regression and the stacking ensemble indicates the ensemble's predictions are dominated by its Linear Regression base learner and meta-learner, with the XGBoost component contributing negligible additional correction on this dataset. **Impact on Model:** This is the first of several results in this chapter showing that ensembling did not yield a practically meaningful improvement over the simpler linear baseline for this specific feature set and dataset.

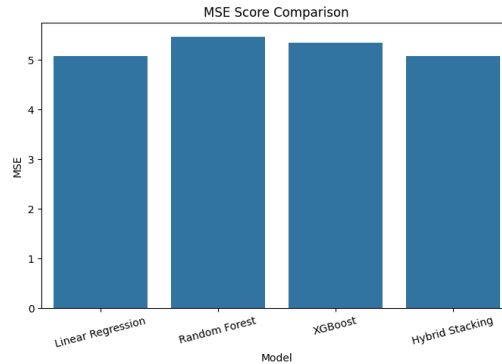


Figure 5.2: MSE Comparison Across Models

Observation: The pattern mirrors the MAE comparison: Linear Regression (5.077) and Hybrid Stacking (5.082) are closely matched and both outperform Random Forest (5.468) and XGBoost (5.342) on this squared-error metric. **Inference:** Because MSE penalizes larger errors more heavily, its similarity between Linear Regression and Stacking further confirms that the two models make comparably-sized errors across the full range of test predictions, including any outliers. **Impact on Model:** The consistency between MAE and MSE findings strengthens confidence that the near-tie between Linear Regression and Stacking is a genuine result rather than a metric-specific artefact.

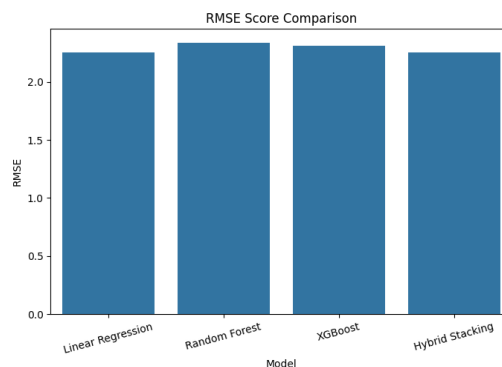


Figure 5.3: RMSE Comparison Across Models

Observation: RMSE values follow the same ranking: Linear Regression (2.253) and Hybrid Stacking (2.254) are virtually indistinguishable, with Random Forest (2.338) and XGBoost (2.311) trailing behind. **Inference:** An RMSE of approximately 2.25 marks means the typical prediction error, weighted toward larger deviations, is just over two marks on the Exam_Score scale — a reasonably tight margin given the target's standard deviation of 3.89. **Impact on Model:** The stacking model's inability to reduce RMSE below the Linear Regression baseline reinforces that additional model complexity was not rewarded on this dataset.

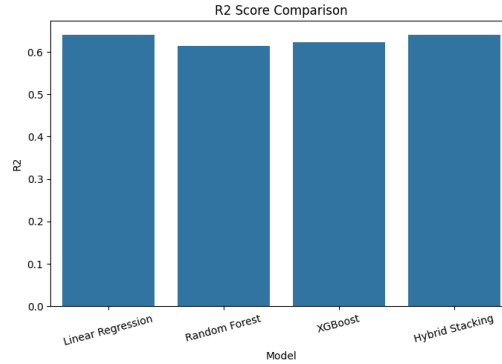


Figure 5.4: R^2 Score Comparison Across Models

Observation: Linear Regression achieves the highest R^2 (0.6408), narrowly ahead of Hybrid Stacking (0.6405). XGBoost (0.6220) and Random Forest (0.6131) explain noticeably less variance. **Inference:** All four models explain roughly 61–64% of the variance in Exam_Score, leaving 36–39% of the variance unexplained by the five available features — likely attributable to unmeasured factors (e.g. the fourteen categorical attributes excluded from modelling, or inherent randomness in exam performance). **Impact on Model:** The R^2 ceiling around 0.64 suggests that further gains are more likely to come from incorporating additional, currently unused features than from further ensembling of the existing five-feature set (see Section 6.4).

5.4 Actual vs. Predicted Visualization

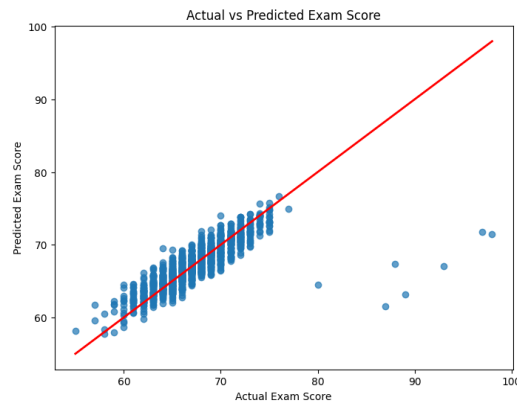


Figure 5.5: Actual vs. Predicted Exam Score (Hybrid Stacking Model)

Observation: Predicted values cluster around the red 45-degree reference line for the bulk of the mid-range scores (roughly 62–72), with increasing scatter and a tendency to under-predict at the higher end of the actual score range. **Inference:** The model performs most reliably for the densely populated middle of the score distribution and is comparatively less reliable for the sparser, higher-scoring students — a common pattern when a training set has limited examples in the tails of the target distribution. **Impact on Model:** This systematic under-prediction at high scores is consistent with the model's moderate R^2 (~ 0.64) and suggests that additional high-scoring examples, or explicit handling of the tail of the distribution, could improve performance for top-performing students specifically.

5.5 Discussion: Why the Hybrid Stacking Model Did Not Outperform Linear Regression

The central, honest finding of this project is that the Hybrid Stacking Regressor did not meaningfully outperform its own Linear Regression base learner on this dataset: R^2 of 0.6405

versus 0.6408, and MAE of 1.2761 versus 1.2751 — differences well within the range of ordinary run-to-run variation. Three factors, evidenced directly by the analysis in this report, explain this result:

- **Predominantly linear feature-target relationships:** The correlation analysis (Table 3.2) and scatter plots (Section 4.3.2) show that Attendance and Hours_Studied — the two dominant predictors — have an approximately linear relationship with Exam_Score, with no strong curvature or threshold effects visible in the pairplot (Figure 4.12). This leaves little non-linear structure for XGBoost to exploit.
- **Low feature-set complexity:** With only five, largely non-multicollinear numeric features (Figure 4.13), the modelling problem is comparatively simple; stacking tends to provide the largest gains when base learners are diverse and each captures materially different, complementary structure in higher-dimensional or more non-linear data.
- **Meta-learner dominated by the linear signal:** Because `passthrough=True` feeds the original (already near-linear) features directly to the Linear Regression meta-learner alongside the two base predictions, and because the Linear Regression base learner's out-of-fold predictions already capture most of the exploitable signal, the meta-learner's final combination closely approximates plain Linear Regression.

This finding is consistent with the independent literature reviewed in Chapter 2: Ahmed et al. (2025), using the same dataset, likewise reported that Linear Regression and Ridge Regression performed exceptionally well as standalone models, with their ensemble Voting Regressor achieving only comparable, not dramatically superior, results ($R^2 = 0.6540$ in their closest feature-selected configuration) — reinforcing that this dataset's Attendance- and Hours_Studied-driven signal is substantially linear in nature.

5.6 Strengths and Limitations

Strengths

- The evaluation methodology is transparent and reproducible: identical train-test split, identical feature set, and four standard metrics were used across all four models.

- The Hybrid Stacking model, while not superior to Linear Regression here, remains competitive and provides a more flexible architecture that could yield gains if additional, more non-linearly related features were introduced.
- Low inter-feature correlation (Figure 4.13) indicates a stable, well-conditioned modelling problem free of severe multicollinearity.

Limitations

- The StandardScaler was fit on the full feature matrix before the train-test split (Section 3.8), introducing a mild, likely small-impact form of data leakage.
- An anomalous Exam_Score value of 100 (Section 3.3, Figure 4.6) exceeds the conventional 0–100 scale and was not explicitly corrected in the source notebook; comparable published work on the same dataset (Ahmed et al., 2025) explicitly treats this value as an outlier and caps it at 100.
- Only five of the twenty available features were used; the fourteen categorical attributes (e.g. Parental_Involvement, Teacher_Quality, School_Type) were excluded from modelling and may carry additional predictive signal.
- R^2 values around 0.64 indicate that roughly 36% of the variance in Exam_Score remains unexplained by the current model, limiting precision for individual, high-stakes predictions.
- The Hybrid Stacking ensemble adds training complexity and inference overhead relative to Linear Regression without a corresponding accuracy benefit on this particular dataset and feature set.

Chapter 6: Conclusion and Future Scope

6.1 Project Summary

This project developed and evaluated an AI-Based Student Performance Prediction System using the Student Performance Factors dataset (6,607 records, 20 attributes). Five numeric features — Attendance, Hours_Studied, Previous_Academic_Scores_Avg, Sleep_Hours, and Tutoring_Sessions — were used to predict Exam_Score through four regression models: Linear Regression, Random Forest, XGBoost, and a Hybrid Stacking Regressor combining Linear Regression and XGBoost with a Linear Regression meta-learner (passthrough=True). The trained pipeline was serialized and deployed as an interactive Streamlit dashboard providing real-time predictions, performance categorization, and personalized recommendations.

6.2 Achievements

- A complete, reproducible pipeline was built spanning data cleaning, EDA, feature scaling, model training, evaluation, persistence, and deployment.
- Four regression models were rigorously compared on an identical train-test split using MAE, MSE, RMSE, and R^2 Score, achieving a best R^2 of 0.6408 (Linear Regression) and 0.6405 (Hybrid Stacking).
- An honest, evidence-based discussion was provided explaining why the stacking ensemble did not outperform its Linear Regression base learner on this dataset, grounded in correlation analysis and visual EDA rather than assumption.
- The trained model was successfully deployed as a working, interactive Streamlit application with real-time inference and rich visual analytics.

6.3 Limitations

- The current model uses only five of the twenty available features, leaving potentially useful categorical information (parental involvement, teacher quality, school type, etc.) unused.
- An anomalous target value (Exam_Score = 100) was not corrected, and the scaler was fit before the train-test split — both documented transparently in Chapter 5.
- With $R^2 \approx 0.64$, the model explains roughly two-thirds of the variance in exam outcomes, leaving meaningful room for improvement before the tool could be relied upon for high-stakes, individual-level decisions.

6.4 Real-World Applications

- Early-warning dashboards for academic advisors to flag students likely to underperform.
- Personalized study-habit and tutoring recommendations generated automatically from a student's profile.
- Institutional analytics for tracking how attendance and study-hour policies correlate with cohort outcomes over time.

6.5 Future Improvements

Batch Prediction

Extending the Streamlit application to accept a CSV upload of multiple students and return predictions for an entire class or cohort in a single operation.

REST API

Exposing the trained pipeline through a FastAPI-based REST endpoint would allow third-party systems (e.g. institutional Student Information Systems) to request predictions programmatically.

Authentication

Adding user authentication would allow the system to store and track individual students' prediction history over time, enabling longitudinal trend analysis.

Explainable AI (SHAP/LIME)

Integrating SHAP or LIME, as demonstrated in comparable published work on this same dataset (Section 2.4), would let the application explain which specific features drove an individual student's prediction, increasing trust and actionability for educators.

Cloud Deployment

Deploying the application on a managed cloud platform (e.g. Streamlit Community Cloud, AWS, or Azure) would provide public accessibility and improved reliability over local execution.

Database Integration

Connecting the application to a persistent database would allow prediction logs, student records, and recommendation histories to be stored and queried over time rather than existing only within a single session.

Mobile Support

A responsive or dedicated mobile interface would extend the tool's reach to educators and students who primarily use mobile devices.

Additional Feature Incorporation

Incorporating the fourteen currently unused categorical attributes (Parental_Involvement, Teacher_Quality, School_Type, etc.) through appropriate encoding, and correcting the anomalous Exam_Score outlier identified in Section 5.6, are the most direct and evidence-backed next steps for improving predictive accuracy beyond the current R^2 ceiling of approximately 0.64.

6.6 Concluding Remarks

This project demonstrates that a complete, end-to-end machine learning system — from raw data to a deployed, interactive application — can be built and rigorously evaluated using standard, well-documented tools. Perhaps its most valuable lesson is a methodological one: the Hybrid Stacking Regressor, despite its added architectural sophistication, did not outperform a plain Linear Regression baseline on this dataset, a result confirmed by evaluation metrics, visual EDA, and consistency with independent published research on the same data. Reporting this honestly, rather than presenting only favourable outcomes, reflects sound data science practice and provides a firmer, more trustworthy foundation for the future improvements outlined above.

References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [Online]. Available: <https://xgboost.readthedocs.io/>
- [3] Streamlit Inc., “Streamlit Documentation,” 2024. [Online]. Available: <https://docs.streamlit.io/>
- [4] Plotly Technologies Inc., “Plotly Python Open Source Graphing Library,” 2024. [Online]. Available: <https://plotly.com/python/>
- [5] Lai Ng, “Student Performance Factors Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/lainguyn123/student-performance-factors>
- [6] W. Ahmed, M. A. Wani, P. Plawiak, S. Meshoul, A. Mahmoud, and M. Hammad, “Machine learning-based academic performance prediction with explainability for enhanced decision-making in educational institutions,” *Scientific Reports*, vol. 15, art. 26879, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-12353-4>
- [7] B. Tang, S. Li, and C. Zhao, “Predicting the Performance of Students Using Deep Ensemble Learning,” *Journal of Intelligence*, vol. 12, no. 12, art. 124, 2024. [Online]. Available: <https://doi.org/10.3390/jintelligence12120124>
- [8] Y. Wang, A. Ding, K. Guan, S. Wu, and Y. Du, “Graph-based Ensemble Machine Learning for Student Performance Prediction,” *arXiv:2112.07893*, 2021. [Online]. Available: <https://arxiv.org/abs/2112.07893>
- [9] M. M. Rahman et al., “A customized ensemble machine learning approach: predicting students’ exam performance,” *Cogent Education*, art. 2490528, 2025. [Online]. Available: <https://doi.org/10.1080/23311916.2025.2490528>
- [10] A. F. A. Sayed, M. A. Arafa, N. A. El-Nimr, K. A. S. Banawan, and M. S. Abdou, “Comparison of machine learning classification and regression models for prediction of academic performance among postgraduate public health students,” *Scientific Reports*, 2025. [Online]. Available: <https://doi.org/10.1038/s41598-025-31023-z>

- [11] E. A. Aldhahri, A. A. Almazroi, and N. Ayub, “Regularized multi-path XSENet ensembler for enhanced student performance prediction in higher education,” *PeerJ Computer Science*, 2025. [Online]. Available: <https://doi.org/10.7717/peerj-cs.3032>
- [12] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61. [Online]. Available: <https://pandas.pydata.org/>